



INTERFACE SPECIFICATION DOCUMENT (ISD)

Document 01 of 09 · B2B Data Engineering Platform



Defines how each source system hands data to the platform: objects, formats, frequency, extraction and load behaviour, controls and error handling.

Document ID	B2B-DEP-01
Version	1.3
Status	Final
Date	05/10/2026
Prepared by	Syed Kazim Raza
Reviewed by	[Team Lead]

05/10/2026

DATAROPES.AI

Data Engineering Internship · B2B Data Engineering Platform

Document Control

Revision History

Version	Date	Author	Description of Change
0.1	-	Syed Kazim Raza	Template created
1.0	29/09/2026	Syed Kazim Raza	Content populated from pipeline code, source profiling and pipeline_run_log
1.1	30/09/2026	Syed Kazim Raza	Updated to the rebuilt data; CSV TRUNCATE + load in one transaction; staging emptied after each run; quarantine
1.2	04/10/2026	Syed Kazim Raza	Interface diagram redrawn (light theme, editable draw.io source in docs/diagrams)
1.3	05/10/2026	Syed Kazim Raza	Formatting to documentation standard: table column widths, numbered figure and table captions with lists, figure descriptions, complete acronym list

Approvals

Role	Name	Signature	Date
Prepared by	Syed Kazim Raza		
Reviewed by	[Team Lead]		
Approved by	[Manager]		

Related Documents

Ref	Document	Location
R1	Business Requirements Document (BRD)	Provided by DataRopes
R2	Project Document (technical reference)	docs/deliverables/project_document.md
R3	Project Plan	docs/deliverables/project_plan.md
R4	README / Deployment Guide	README.md

Contents

Document Control	2
Contents	3
List of Figures.....	4
List of Tables.....	5
1. Introduction.....	6
1.1 Purpose.....	6
1.2 Scope	6
1.3 Intended Audience	6
1.4 Definitions & Acronyms.....	7
2. Interface Overview	8
3. Source System Details	9
4. Interface IF-01: SQL Server → staging (8 tables)	9
4.1 General Information.....	9
4.2 Objects Transferred	10
4.3 Extraction & Load Logic	10
4.4 Error Handling & Re-run	10
5. Interface IF-02: marketing_leads.csv → staging.marketing_leads.....	11
5.1 General Information	11
5.2 File Specification.....	11
5.3 Record Layout.....	11
5.4 Extraction & Load Logic	12
5.5 Error Handling & Re-run	12
6. Interface IF-03: web_logs.csv → staging.web_logs.....	14
6.1 General Information.....	14
6.2 File Specification.....	14
6.3 Record Layout.....	14
6.4 Extraction & Load Logic	15
6.5 Error Handling & Re-run	15
7. Scheduling & Orchestration.....	17
8. Control & Audit Tables	18
9. Interface Data Quality Checks	19
10. Error Handling & Recovery	20
11. Security	20
12. Assumptions, Constraints & Open Issues	21

List of Figures

Figure 1: Interface diagram — sources, interfaces and staging targets..... 8

Figure 2: Sample rows of marketing_leads.csv 13

Figure 3: Sample rows of web_logs.csv..... 16

Figure 4: Staging tables are empty after a successful run..... 17

Figure 5: metadata.pipeline_run_log — one row per pipeline stage and run 18

Figure 6: metadata.pipeline_watermarks — one watermark per SQL Server table 19

List of Tables

Table 1: Intended Audience..... 6

Table 2: Definitions & Acronyms 7

Table 3: Interface inventory 9

Table 4: Source System Details..... 9

Table 5: IF-01 general information 9

Table 6: IF-01 objects transferred 10

Table 7: IF-02 general information 11

Table 8: IF-02 file specification 11

Table 9: IF-02 record layout..... 11

Table 10: IF-03 general information 14

Table 11: IF-03 file specification 14

Table 12: IF-03 record layout..... 14

Table 13: Scheduling & Orchestration..... 17

Table 14: Control & Audit Tables..... 18

Table 15: Interface Data Quality Checks 19

Table 16: Error Handling & Recovery 20

Table 17: Assumptions, Constraints & Open Issues 21

1. Introduction

1.1 Purpose

This document specifies every data interface between the source systems and the PostgreSQL staging layer of the B2B Data Engineering Platform.

1.2 Scope

In scope:

- Interfaces from SQL Server (b2b_source_db) and the two CSV feeds into PostgreSQL staging
- Scheduling, control tables, error handling and security of those interfaces

Out of scope:

- Transformations after staging (see SMX/SMD)
- Data generation logic (see Development Document)

1.3 Intended Audience

Table 1: Intended Audience

Audience	How they use this document
Data Engineers	Build and maintain the ingestion pipelines
Team Lead / Reviewer	Confirm interface behaviour matches the BRD
QA / Testers	Derive interface test cases

1.4 Definitions & Acronyms

Table 2: Definitions & Acronyms

Term	Definition
B2B	Business-to-Business: companies selling to other companies.
BRD	Business Requirements Document: here the Project Requirements Document provided by DataRopes.
CDC	Change Data Capture: extracting only records that are new or changed since the last run.
CI/CD	Continuous Integration / Continuous Delivery: automated build, test and deployment (out of scope for this project).
CSV	Comma-Separated Values: the text-file format of the marketing leads and web logs feeds.
DEP	Data Engineering Platform: prefix of the document IDs (B2B-DEP-01 ...).
DQ	Data Quality: DQ-x numbers the data-quality findings of the source analysis.
HTTP	Hypertext Transfer Protocol: the web protocol recorded in the web logs (GET, POST ...).
ID	Identifier.
Interface (IF)	A defined data flow from one source system to its staging target(s), with its own pipeline script.
ISD	Interface Specification Document (deliverable 01).
MB	Megabyte.
ODBC	Open Database Connectivity: the driver standard Python uses to connect to SQL Server.
PII	Personally Identifiable Information: data that identifies a person (names, e-mails, phone numbers).
PK	Primary Key: the column(s) that uniquely identify a row.
Run log	metadata.pipeline_run_log: one row per pipeline execution with status, rows and watermark.
SLA	Service Level Agreement: an agreed target for availability or delivery time.
SMD	Source Mapping Document (deliverable 05, Word).
SMX	Source-to-Target Mapping workbook (deliverable 05, Excel).
Snapshot load	Full re-load of a source (TRUNCATE + append) instead of an incremental extract.
SQL	Structured Query Language.
Staging	PostgreSQL landing schema: raw copy of the latest batch, no keys, plus _loaded_at; emptied after each successful run.
URL	Uniform Resource Locator: a web address.
UTF-8	Unicode character encoding used by the CSV files.
UTM	Urchin Tracking Module: campaign parameters (utm_source, utm_medium, utm_campaign).
Watermark	Highest updated_at successfully extracted per table; stored in metadata.pipeline_watermarks.

2. Interface Overview

Figure 1 gives an overview of the three interfaces: each source, the Python pipeline that moves it and the staging table it lands in.

Interface Specification (ISD) — how each source hands data to the platform

3 interfaces · orchestrated by run_pipeline.py (initial = 2-year backfill, incremental = one simulated day) · every run logged to metadata

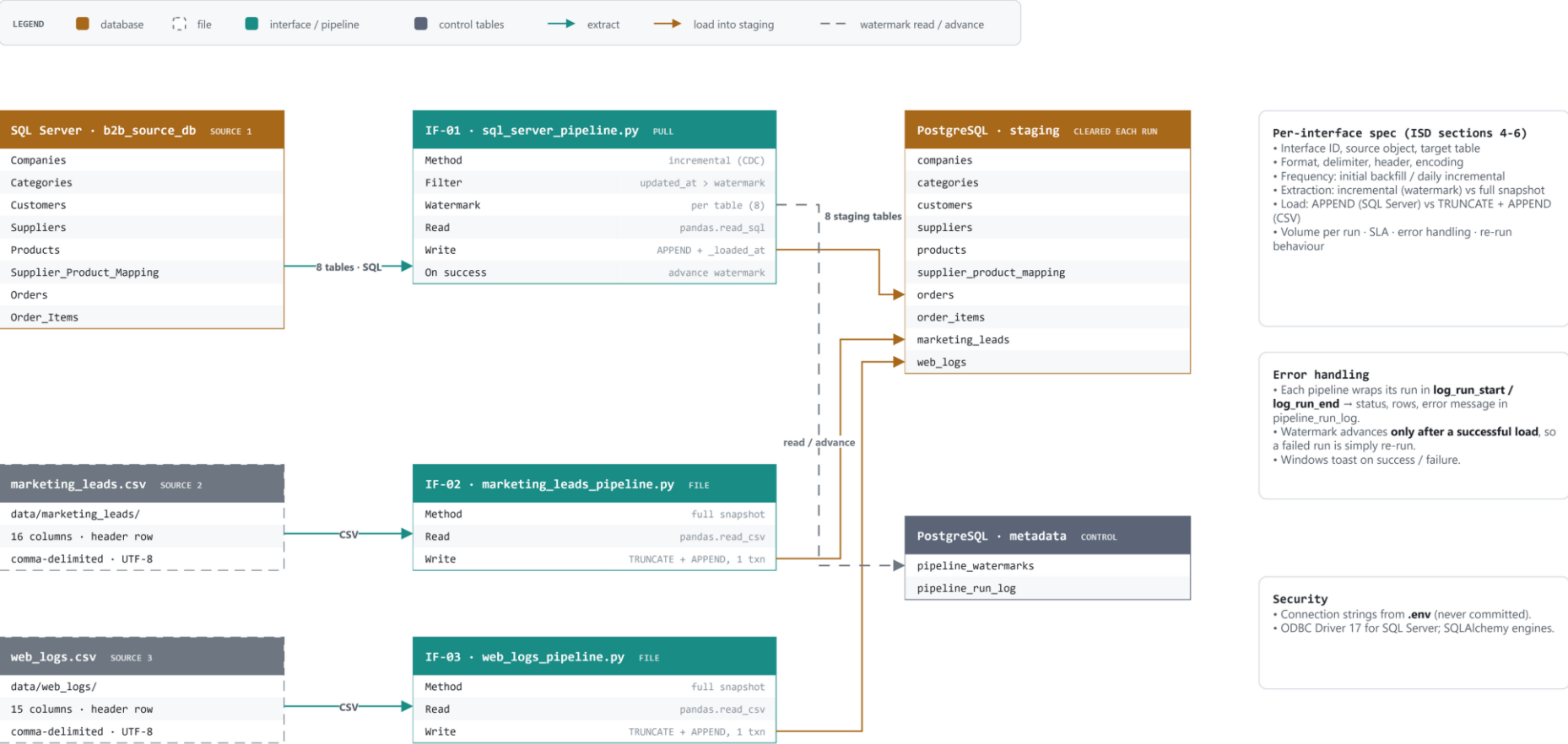


Figure 1: Interface diagram — sources, interfaces and staging targets

Table 3: Interface inventory

Interface ID	Source System	Source Object(s)	Format	Target (staging)	Extraction	Load Method	Frequency
IF-01	SQL Server b2b_source_db	8 tables in schema source (Companies ... Order_Items)	Relational (ODBC)	staging.<table> (8 tables)	Incremental (watermark on updated_at); full on first run	APPEND	Every pipeline run
IF-02	Marketing (CSV feed)	marketing_leads.csv	CSV, comma, header	staging.marketing_leads	Full snapshot	TRUNCATE + APPEND	Every pipeline run
IF-03	Web server (CSV feed)	web_logs.csv	CSV, comma, header	staging.web_logs	Full snapshot	TRUNCATE + APPEND	Every pipeline run

3. Source System Details

Table 4: Source System Details

Source	Technology	Location / Connection	Access Method	Credentials Handling	Owner
S1 SQL Server	Microsoft SQL Server 16.0.1000.6 (2022 Express)	Local instance, database b2b_source_db, schema source. SQLAlchemy URL mssql+pyodbc with the ODBC driver named in .env	Read-only SELECT via pandas.read_sql	Windows authentication; server, driver, database and auth option read from .env (SQL_SERVER_*)	Source system (simulated by python/generators)
S2 Marketing CSV	CSV file on local disk	data/marketing_leads/marketing_leads.csv (path in config/config.py)	pandas.read_csv	File-system permissions only	Marketing (simulated)
S3 Web logs CSV	CSV file on local disk	data/web_logs/web_logs.csv (path in config/config.py)	pandas.read_csv	File-system permissions only	Web platform (simulated)

4. Interface IF-01: SQL Server → staging (8 tables)

4.1 General Information

Table 5: IF-01 general information

Interface ID	IF-01
Interface name	SQL Server → staging (8 tables)
Source	SQL Server b2b_source_db, schema source (8 tables)
Target	PostgreSQL b2b_warehouse_db, schema staging (8 tables of the same name, lower case)
Pipeline script	python/pipelines/sql_server_pipeline.py (queries in python/utils/constants.py SOURCE1_QUERIES)
Trigger / orchestration	python -m python.pipelines.run_pipeline initial incremental (started manually from the command line)
Frequency	Initial backfill once; then one incremental run per simulated business day
Extraction type	Incremental per table (watermark); full extract when no watermark exists yet
Change indicator / watermark	updated_at; key sql_server_<table> in metadata.pipeline_watermarks
Load type	APPEND (DataFrame.to_sql, chunksize 2,500)
Expected volume per run	Initial: 722,463 rows. Incremental: about 850-900 new/changed rows (run log: min 851 rows extracted)
SLA / completion window	Measured: average 21.2 s, maximum 78.8 s per run (4 completed runs; the maximum is the initial load)

4.2 Objects Transferred

Table 6: IF-01 objects transferred

#	Source Table	Target Table	Watermark Column	Initial Rows	Rows / Day	Load Order
1	source.Companies	staging.companies	updated_at	2,500	1-3 updated	Level 0
2	source.Categories	staging.categories	updated_at	10	0	Level 0
3	source.Customers	staging.customers	updated_at	94,755	3-5 updated	Level 1
4	source.Suppliers	staging.suppliers	updated_at	400	0	Level 1
5	source.Products	staging.products	updated_at	96	0-2 deactivated	Level 1
6	source.Supplier_Product_Mapping	staging.supplier_product_mapping	updated_at	347	0	Level 2
7	source.Orders	staging.orders	updated_at	156,678	180-230 new + 5-10 status updates	Level 3
8	source.Order_Items	staging.order_items	updated_at	470,263	~550-700 new (lines of new orders)	Level 4

4.3 Extraction & Load Logic

One execution of `sql_server_pipeline.run()` performs these steps:

1. **Log start:** insert a row in `metadata.pipeline_run_log` with status 'running' and a new `run_id`.
2. **Extract:** read the table's watermark; if it exists, `SELECT ... FROM source.<Table> WHERE updated_at > :watermark`; otherwise the full table. The watermark is passed as a bind parameter, not concatenated into the SQL.
3. **Load:** append each non-empty DataFrame to `staging.<table>` with `to_sql(if_exists='append', chunksize=2500)`. PostgreSQL fills `_loaded_at` with `now()`.
4. **Advance watermarks:** only after all loads succeeded, store `MAX(updated_at)` of each table whose watermark moved.
5. **Log end:** update the run-log row with status 'completed', `rows_extracted`, `rows_loaded` and the highest watermark.

4.4 Error Handling & Re-run

- Any exception is caught in `run()`: the run-log row is closed with status 'failed' and the error message, and the exception is re-raised so `run_pipeline` stops and shows a Windows failure notification.
- Watermarks are only advanced after a successful load, so a failed run changes no watermark. The next run re-extracts the same rows.
- If a failure happens after some tables were already appended, those rows are appended again on the re-run. This is safe: the intermediate step keeps only the latest version per key, so duplicates are removed.
- Re-run: simply execute the same command again; no manual clean-up is required.

5. Interface IF-02: marketing_leads.csv → staging.marketing_leads

5.1 General Information

Table 7: IF-02 general information

Interface ID	IF-02
Interface name	marketing_leads.csv → staging.marketing_leads
Source	CSV file data/marketing_leads/marketing_leads.csv
Target	PostgreSQL staging.marketing_leads
Pipeline script	python/pipelines/marketing_leads_pipeline.py
Trigger / orchestration	python -m python.pipelines.run_pipeline initial incremental (manual command line)
Frequency	Every pipeline run
Extraction type	Full snapshot (whole file is read every run)
Change indicator / watermark	None - the file has updated_at, but the file is rewritten in place when leads change, so it is reloaded completely
Load type	TRUNCATE staging table + APPEND (to_sql)
Expected volume per run	Whole file: currently 150,616 rows (31.7 MB), growing by 180-230 leads per day
SLA / completion window	Measured: average 17.7 s, max 20.7 s per run

5.2 File Specification

Table 8: IF-02 file specification

File name / path	data/marketing_leads/marketing_leads.csv
Naming convention	Fixed name marketing_leads.csv (single cumulative file, no date suffix)
Format	CSV
Delimiter	Comma (,)
Quote character	Double quote (") - pandas default, only when a value contains a comma
Header row	Yes - first row holds the column names
Encoding	UTF-8
Date / timestamp format	YYYY-MM-DD HH:MM:SS, optionally with microseconds (.ffffff)
Record count / file size	150,616 records, 31.7 MB (profiled 30/09/2026); no trailer/control record

5.3 Record Layout

Table 9: IF-02 record layout

#	Field	Data Type	Length	Nullable	Description	Sample Value
1	lead_id	Text (hex id)	32	N	Identifier of the marketing lead that generated the record (NULL if none).	3ecb55e9...
2	source	Text	100	N	Marketing channel the lead came from (e.g. Website, Referral).	Cold Call
3	campaign_name	Text	150	N	Marketing campaign name.	Summer Sale
4	utm_source	Text	100	N	UTM source parameter (e.g. google, linkedin).	linkedin
5	utm_medium	Text	100	N	UTM medium parameter (e.g. cpc, email).	social
6	utm_campaign	Text	150	N	UTM campaign parameter.	summer_sale
7	company_name	Text	200	N	Legal/trading name of the company.	Beck-Walsh
8	company_size	Text	30	N	Employee-size band of the lead's company.	1-10
9	industry	Text	100	N	Industry of the lead's company.	Technology
10	country	Text	100	N	Country name.	United States
11	city	Text	100	N	City name.	Seattle
12	lead_score	Integer	-	N	Lead quality score, 1-100.	7
13	estimated_order_value	Decimal	12,2	N	Estimated value of the potential order.	5747.47
14	funnel_stage	Text	30	N	Current sales-funnel stage of the lead.	New
15	created_at	Timestamp	-	N	Timestamp when the record was created in the source system.	2025-08-08 05:02:55

#	Field	Data Type	Length	Nullable	Description	Sample Value
16	updated_at	Timestamp	-	N	Timestamp of the last change in the source system; drives incremental extraction and upserts.	2026-03-21 06:15:54

5.4 Extraction & Load Logic

1. **Log start** in metadata.pipeline_run_log (status 'running').
2. **Read** the whole file with `pd.read_csv(..., parse_dates=[created_at, updated_at])`; other types are inferred by pandas.
3. **Profile** the DataFrame (shape, dtypes, null counts) to the log.
4. **Load**: TRUNCATE staging.marketing_leads and to_sql(if_exists='append', chunksize=2500) in one transaction. _loaded_at is filled by PostgreSQL.
5. **Log end** with status 'completed' and the row count. No watermark is stored for this interface.

How the file grows: the daily generator appends new leads to the file and rewrites existing rows when a lead's funnel_stage / lead_score changes, so the file always holds the current state of every lead.

5.5 Error Handling & Re-run

- A missing or unreadable file, or a parse error, fails the run before the TRUNCATE, so staging keeps the previous snapshot.
- If the append fails, the TRUNCATE is rolled back too, so staging keeps the previous snapshot. The pipeline stops and the transform does not run.
- Failures are logged with status 'failed' and the error message, and run_pipeline shows a failure notification.
- Re-run is always safe: the interface is a full snapshot, so running it again produces exactly the same staging content.

Figure 2 shows the first rows of the marketing leads file as delivered, with its header row and comma-separated layout.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	lead_id	source	campaign_name	utm_source	utm_medium	utm_campaign	company_name	company_size	industry	country	city	lead_score	estimated_order_value	funnel_stage	created_at	updated_at
2	3ecb55e90827474a8623121de0bbf37	Cold Call	Summer Sale	linkedin	social	summer_sale	Beck-Walsh	1-Oct	Technology	United States	Seattle	7	5747.47	New	21/9/2025 5:02	4/5/2026 6:15
3	e137404546f1417099300965c313065	Website	Industry Expo	facebook	referral	industry_expo	Alexander, Johnson and Kemp	201-500	Education	United Kingdom	London	15	189761.32	New	9/3/2025 6:20	16/6/2025 23:22
4	b505d6e19e9c4914b3b52c97a42a03f	Website	Referral Drive	linkedin	email	referral_drive	Horn-Brown	Nov-50	Retail	Australia	Sydney	20	39506.8	Contacted	8/1/2025 21:01	23/2/2025 22:37
5	5336f85ef7314abf801e4da81d15230f	Referral	Product Launch	newsletter	cpc	product_launch	Harrington Ltd	Nov-50	Logistics	United States	Seattle	42	27336.7	Qualified	7/6/2026 12:07	7/8/2026 7:47
6	684a78f718fa4906b9d00763e21ca72	Email Campaign	Enterprise Growth	linkedin	organic	enterprise_growth	Kirby LLC	1-Oct	Retail	United States	Seattle	33	8569.17	Qualified	8/7/2026 17:05	28/8/2026 12:40
7	2760dfe5bcd7430fba2172d255ffa25e	Trade Show	Holiday Promotion	linkedin	referral	holiday_promotion	Martin-Parker	51-200	Hospitality	United States	New York	43	148834.24	Lost	20/4/2026 17:43	14/9/2026 15:10
8	28714edefd314d7d88d533807639e85f	Partner	Referral Drive	bing	organic	referral_drive	Clark, Hamilton and Brown	1-Oct	Construction	United States	New York	13	2706.87	New	3/7/2026 18:22	1/8/2026 1:22
9	024a88bd95d744da8b3a67989bf991f	Partner	Free Trial Campaign	facebook	cpc	free_trial_campaign	Mcdonald LLC	Nov-50	Finance	United States	Austin	72	22210.9	Negotiation	27/12/2025 9:20	16/3/2026 23:56
10	7425579e761a420abdab60b0d74d0b	Website	Enterprise Growth	facebook	cpc	enterprise_growth	Ruiz Ltd	51-200	Technology	United States	New York	54	130368.2	Proposal	26/12/2024 4:46	23/5/2025 15:26
11	48f74b1a740342c6adf9fa249f2c978b	Email Campaign	Industry Expo	facebook	organic	industry_expo	Gaines, Mcneil and Wright	Nov-50	Construction	United States	Austin	32	47865.4	Contacted	9/8/2026 5:34	10/9/2026 10:40
12	2fd55742628746ae9b72ccad260f853f	Website	Industry Expo	linkedin	email	industry_expo	Griffin, Randall and Perry	51-200	Construction	United States	Chicago	74	115191.51	Negotiation	28/10/2024 20:14	16/7/2026 17:31
13	e8eba2a584f8428abaae04a36b0910c	Cold Call	Free Trial Campaign	newsletter	organic	free_trial_campaign	Bullock Group	Nov-50	Energy	United Kingdom	Manchester	41	47042.67	Qualified	30/12/2025 2:37	6/1/2026 14:37
14	f55798cc4e2647f7a8dbad5c2bda20c	Email Campaign	Free Trial Campaign	newsletter	referral	free_trial_campaign	Lambert-Thompson	1-Oct	Technology	United States	New York	20	3748.18	Contacted	30/5/2026 7:30	6/7/2026 22:11
15	3dee0eb80eb94a50bfc9e24cc5d648f6	Partner	Enterprise Growth	linkedin	cpc	enterprise_growth	Brown PLC	1-Oct	Education	United States	New York	49	9180.49	Proposal	19/6/2025 0:47	23/10/2025 20:00

Figure 2: Sample rows of marketing_leads.csv

6. Interface IF-03: web_logs.csv → staging.web_logs

6.1 General Information

Table 10: IF-03 general information

Interface ID	IF-03
Interface name	web_logs.csv → staging.web_logs
Source	CSV file data/web_logs/web_logs.csv
Target	PostgreSQL staging.web_logs
Pipeline script	python/pipelines/web_logs_pipeline.py
Trigger / orchestration	python -m python.pipelines.run_pipeline initial incremental (manual command line)
Frequency	Every pipeline run
Extraction type	Full snapshot (whole file is read every run)
Change indicator / watermark	None - log lines are immutable and the file has no change column
Load type	TRUNCATE staging table + APPEND (to_sql)
Expected volume per run	Whole file: currently 301,220 rows (59.9 MB), growing by 370-450 log lines per day
SLA / completion window	Measured: average 30.0 s, max 41.0 s per run

6.2 File Specification

Table 11: IF-03 file specification

File name / path	data/web_logs/web_logs.csv
Naming convention	Fixed name web_logs.csv (single cumulative file, no date suffix)
Format	CSV
Delimiter	Comma (,)
Quote character	Double quote (") - pandas default, only when a value contains a comma
Header row	Yes - first row holds the column names
Encoding	UTF-8
Date / timestamp format	YYYY-MM-DD HH:MM:SS, optionally with microseconds (.ffffff)
Record count / file size	301,220 records, 59.9 MB (profiled 30/09/2026); no trailer/control record

6.3 Record Layout

Table 12: IF-03 record layout

#	Field	Data Type	Length	Nullable	Description	Sample Value
1	log_id	Text (hex id)	32	N	Identifier of the web request log entry.	c4896ec9...
2	country	Text	100	N	Country name.	United Kingdom
3	city	Text	100	N	City name.	Manchester
4	log_timestamp	Timestamp	-	N	Timestamp of the web request.	2026-01-13 13:18:50
5	client_ip	Text	45	N	IP address of the client.	10.19.246.214
6	auth_user	Text	150	Y (15,014 empty)	Authenticated user name (NULL for anonymous requests).	martyn70
7	session_id	Text (hex id)	32	N	Web session identifier.	f831053e...
8	http_method	Text	10	N	HTTP method of the request (GET, POST, PUT, DELETE).	DELETE
9	request_path	Text	255	N	Requested URL path.	/api/orders
10	status_code	Integer	-	N	HTTP response status code (100-599).	200
11	bytes_sent	Integer	-	N	Response size in bytes.	356429
12	referrer	Text	255	N	Referring URL.	https://twitter.com
13	device_type	Text	30	N	Client device type: Desktop, Mobile or Tablet.	Desktop
14	browser	Text	50	N	Client browser.	Firefox
15	is_bot	Boolean (True/False)	-	N	TRUE if the request came from a bot / crawler.	False

6.4 Extraction & Load Logic

1. **Log start** in metadata.pipeline_run_log (status 'running').
2. **Read** the whole file with `pd.read_csv(..., parse_dates=[log_timestamp])`; other types are inferred by pandas.
3. **Profile** the DataFrame (shape, dtypes, null counts) to the log.
4. **Load**: TRUNCATE staging.web_logs and to_sql(if_exists='append', chunksize=2500) in one transaction. _loaded_at is filled by PostgreSQL.
5. **Log end** with status 'completed' and the row count. No watermark is stored for this interface.

How the file grows: the daily generator appends new request lines (append-only); existing lines never change.

6.5 Error Handling & Re-run

- A missing or unreadable file, or a parse error, fails the run before the TRUNCATE, so staging keeps the previous snapshot.
- If the append fails, the TRUNCATE is rolled back too, so staging keeps the previous snapshot. The pipeline stops and the transform does not run.
- Failures are logged with status 'failed' and the error message, and run_pipeline shows a failure notification.
- Re-run is always safe: the interface is a full snapshot, so running it again produces exactly the same staging content.

Figure 3 shows the first rows of the web logs file as delivered.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	log_id	country	city	log_timestamp	client_ip	auth_user	session_id	http_method	request_path	status_code	bytes_sent	referrer	device_type	browser	is_bot
2	c4896ec91b654828a96b4bcc101eb26	United Kingdom	Manchester	26/2/2026 13:18	10.19.246.214	martyn70	f831053ec82048b09d43ffb9b3168c59	DELETE	/api/orders	200	356429	https://twitter.com	Desktop	Firefox	FALSE
3	f102f7333a9a4c329e36194897e4aeb6	United States	New York	5/3/2025 7:04	10.19.246.214	joshua70	304f5c647d0d43a98dde90cb0443f5cc	DELETE	/api/customers	200	191499	https://twitter.com	Mobile	Edge	FALSE
4	53fed5ce0d4d4591b7d9dcf78732dc67	United Kingdom	London	9/8/2026 8:52	101.221.232.233	bethany29	8001a630e3b0471da202422a6925c172	DELETE	/api/customers	400	854	https://bing.com	Desktop	Edge	FALSE
5	90012112126444f2948b514b9b6df45a	United Kingdom	Birmingham	29/5/2026 10:48	166.40.114.106	brianhayward	e9d717b039ad4ff2884591741d4430f1	PUT	/checkout	403	3609	https://twitter.com	Desktop	Chrome	FALSE
6	c3da80ca65ad433fbb20bcb207af9d94	United States	Seattle	6/3/2025 15:31	101.221.232.233	steven29	7c2d551a5dee4edfb2bde2896160534d	GET	/api/customers	200	433619	https://bing.com	Mobile	Chrome	FALSE
7	a373826495c144d1a50a41c10a518b19	United States	Seattle	1/5/2025 9:18	110.40.114.106	chadjohnson	b3dee6ff12504ae18993f48d68327b3d	POST	/checkout	200	123227	https://twitter.com	Desktop	Chrome	FALSE
8	2e72cdd13e5a438bb8b7b04b257c8713	Australia	Sydney	4/2/2025 12:23	10.19.246.214	joshua70	53169ea0498445ea9d10417e16a69bec	POST	/products/123	200	290857	https://google.com	Desktop	Chrome	FALSE
9	ebdd36c7c4564deb96ca17832a74eeba	United States	Austin	13/10/2025 13:40	200.42.216.22	owenkatherine	39c02eacfa472caa1f77d783a49033	GET	/products/123	301	474996	https://bing.com	Mobile	Chrome	FALSE
10	c1f8343ebc534032b4aca41ebef0fdd1	United States	Seattle	2/3/2025 15:57	171.108.1.47	brendajohnson	d8490c7e4a504d199ef80ca76885860e	PUT	/api/customers	400	4267	https://twitter.com	Desktop	Safari	FALSE
11	90f6b1e554c34661a605461d14c20b4b	United Kingdom	London	10/7/2025 14:26	200.42.216.22	robertstracey	f0250e4db7b340248133e469b48825a1	PUT	/checkout	200	333133	https://twitter.com	Desktop	Chrome	FALSE
12	d35520bd477243d690d4638eada06f5a	United States	Dallas	3/8/2025 9:20	27.113.139.219	harrisvictoria	502145f3ccce4e71add490bb27fc7840	DELETE	/checkout	200	224653	https://twitter.com	Tablet	Chrome	FALSE
13	a4c8e058f3124d5f83ae75054fa05064	United States	Dallas	20/4/2025 21:24	222.22.6.30	cruzchristopher	1b289edb85864a87ab529576e064c65c	DELETE	/login	200	169383	https://twitter.com	Desktop	Chrome	FALSE
14	2175990f5e9640f0a423db332d5f3b90	United States	New York	13/6/2026 14:16	4.236.116.216	ashleymartinez	2255a98876b94b328339afefc5b51397	GET	/login	404	4946	https://google.com	Desktop	Safari	FALSE
15	a2d4aa1da3ed46b4875ea2c6e3db75a4	United States	Chicago	25/7/2026 22:29	81.26.16.200	danielmaldonado	d03507eb0a5846aabac52ec584d0d7d4	PUT	/search?q=laptop	200	79354	https://bing.com	Mobile	Firefox	FALSE

Figure 3: Sample rows of web_logs.csv

7. Scheduling & Orchestration

Table 13: Scheduling & Orchestration

Run Mode	Command	Interfaces Executed	Order	Trigger
initial	python -m python.pipelines.run_pipeline initial	Generators (master + transactions), IF-01, IF-02, IF-03, transform, warehouse load	Generate -> IF-01 -> IF-02 -> IF-03 -> intermediate -> clear staging -> warehouse	Manual, once per environment
incremental	python -m python.pipelines.run_pipeline incremental	CDC generator (one simulated day), IF-01, IF-02, IF-03, transform, warehouse load	Generate day -> IF-01 -> IF-02 -> IF-03 -> intermediate -> clear staging -> warehouse	Manual, once per simulated day (BRD: no CI/CD, local orchestration)

Figure 4 is the evidence that staging is emptied after a successful run: every staging table returns 0 rows.

13

14

15

16

17

18

19

20

21

22

```
SELECT 'orders' AS staging_table, COUNT(*) AS rows_left FROM staging.orders
UNION ALL
SELECT 'order_items', COUNT(*) FROM staging.order_items
UNION ALL
SELECT 'marketing_leads', COUNT(*) FROM staging.marketing_leads
UNION ALL
SELECT 'web_logs', COUNT(*) FROM staging.web_logs
UNION ALL
SELECT 'companies', COUNT(*) FROM staging.companies;
```

Data OutputMessagesNotifications

Showing rows: 1 to 5

	staging_table	rows_left
	text	bigint
1	orders	0
2	order_items	0
3	marketing_leads	0
4	web_logs	0
5	companies	0

Figure 4: Staging tables are empty after a successful run

8. Control & Audit Tables

Table 14: Control & Audit Tables

Control Table	Column	Purpose
metadata.pipeline_watermarks	pipeline_name (PK)	Watermark key: sql_server_<table> (8 rows)
metadata.pipeline_watermarks	last_extracted_at	Highest updated_at loaded successfully; lower bound of the next extract
metadata.pipeline_run_log	run_id, pipeline_name	One row per execution of a pipeline or stage (interfaces, intermediate_transform, warehouse_load)
metadata.pipeline_run_log	run_started_at, run_ended_at, status	Duration and outcome: running / completed / failed
metadata.pipeline_run_log	rows_extracted, rows_loaded, watermark_used	Volumes and watermark of the run, for reconciliation
metadata.pipeline_run_log	error_message	Exception text of a failed run
staging.<every table>	_loaded_at	Ingestion timestamp of each row; used to pick the latest version downstream

Figure 5 shows the run log: one row per pipeline stage and run, with start and end time, rows loaded and status.

Query		Query History				
1	SELECT					
2	pipeline_name, status, rows_extracted, rows_loaded, run_started_at, run_ended_at					
3	FROM metadata.pipeline_run_log					
4	ORDER BY run_started_at DESC					
5	LIMIT 10;					
6						
Data Output		Messages				
		Notifications				
		Showing rows: 1 to 10				
		Page No: 1				
	pipeline_name character varying (100)	status character varying (20)	rows_extracted integer	rows_loaded integer	run_started_at timestamp without time zone	run_ended_at timestamp without time zone
1	warehouse_load	completed	[null]	1501	2026-09-30 17:32:26.594969	2026-09-30 17:33:26.293246
2	intermediate_transform	completed	[null]	297567	2026-09-30 17:31:42.741172	2026-09-30 17:32:25.471447
3	web_logs_pipeline	completed	301220	301220	2026-09-30 17:31:10.245685	2026-09-30 17:31:39.454006
4	marketing_leads_pipeli...	completed	150616	150616	2026-09-30 17:30:49.968991	2026-09-30 17:31:05.837898
5	sql_server_pipeline	completed	881	881	2026-09-30 17:30:43.129156	2026-09-30 17:30:44.729435
6	warehouse_load	completed	[null]	1498	2026-09-30 17:29:30.066425	2026-09-30 17:30:25.780681
7	intermediate_transform	completed	[null]	297165	2026-09-30 17:28:43.258518	2026-09-30 17:29:28.322206
8	web_logs_pipeline	completed	300806	300806	2026-09-30 17:28:16.144543	2026-09-30 17:28:39.986211
9	marketing_leads_pipeli...	completed	150413	150413	2026-09-30 17:27:57.621214	2026-09-30 17:28:11.602063
10	sql_server_pipeline	completed	851	851	2026-09-30 17:27:50.377578	2026-09-30 17:27:52.503623

Figure 5: metadata.pipeline_run_log — one row per pipeline stage and run

Figure 6 shows the stored watermarks, one per SQL Server table; the next incremental run extracts only rows changed after them.

	pipeline_name [PK] character varying (100)	last_extracted_at timestamp without time zone
1	sql_server_categories	2026-09-14 05:10:11
2	sql_server_companies	2026-09-30 17:30:38.062934
3	sql_server_customers	2026-09-30 17:30:38.507897
4	sql_server_order_items	2026-09-30 17:30:37.546816
5	sql_server_orders	2026-09-30 17:30:39.309168
6	sql_server_products	2026-09-30 17:30:41.417926
7	sql_server_supplier_product_mappi...	2026-09-29 23:41:14
8	sql_server_suppliers	2026-09-29 23:06:46

Figure 6: metadata.pipeline_watermarks — one watermark per SQL Server table

9. Interface Data Quality Checks

Table 15: Interface Data Quality Checks

Check ID	Interface	Check	Action on Failure
DQ-01	All	Row count, dtypes and null counts profiled for every extracted DataFrame	Logged for review; load continues
DQ-02	All	rows_extracted = rows_loaded recorded in pipeline_run_log	Mismatch investigated manually
DQ-03	IF-01	Extract filtered by updated_at > watermark; watermark only advanced on success	Failed run leaves watermark unchanged
DQ-04	IF-02, IF-03	File must exist and parse (header, timestamp columns)	Run fails before TRUNCATE; previous snapshot kept
DQ-05	All	Field-level rules (mandatory, domain, range, referential) applied in the intermediate layer - see SMD section 7	Row not promoted; CSV rows saved in intermediate.rejected_records

10. Error Handling & Recovery

Table 16: Error Handling & Recovery

Scenario	Detection	Handling	Recovery / Re-run
SQL Server unreachable / login fails	Exception in read_sql; run log 'failed'; failure notification	Run stops; no watermark moves	Fix connection (.env / service) and re-run
PostgreSQL unreachable	Exception on connect; run stops	Nothing written	Fix and re-run
Failure after partial append (IF-01)	Run log 'failed'	Watermarks unchanged; some tables already appended	Re-run; duplicates removed by intermediate dedup
CSV missing / corrupt	read_csv exception; run log 'failed'	TRUNCATE not executed	Restore/regenerate the file and re-run
Transform or warehouse stage fails	Stage run log 'failed' (e.g. one intermediate_transform failure in the log)	Stage transaction rolled back	Fix cause and re-run the pipeline; all stages are idempotent

11. Security

- **Credentials** are never in code: all connection settings (SQL_SERVER_*, POSTGRES_*) are read from a local .env file with python-dotenv. .env is excluded from Git; the pipeline fails at start-up if any variable is missing.
- **SQL Server** uses Windows authentication (trusted connection) through the ODBC driver, so no SQL Server password is stored.
- **PostgreSQL** uses the user/password from .env.
- **Access:** the pipeline only reads from SQL Server and the CSV files; it writes only to PostgreSQL schemas staging, intermediate, warehouse and metadata.
- **SQL injection:** the watermark is passed as a bound parameter to `text()`; table names come from a fixed mapping, not from user input.
- **Personal data (PII):** customers (name, e-mail, phone, date of birth), supplier contacts, web-log client_ip and auth_user. All data is synthetic (Faker), but in production these columns would require masking or restricted access in the marts.

12. Assumptions, Constraints & Open Issues

Table 17: Assumptions, Constraints & Open Issues

ID	Type	Description	Owner	Status
A1	Assumption	Source updated_at is maintained on every insert and update (required for IF-01 CDC).	Data Engineering	Accepted
A2	Assumption	CSV files always contain the complete current data set (snapshot semantics).	Data Engineering	Accepted
C1	Constraint	Runs are started manually through run_pipeline.py, as specified in the requirements (no CI/CD).	Data Engineering	Accepted